Overview of major embedded Linux software stacks

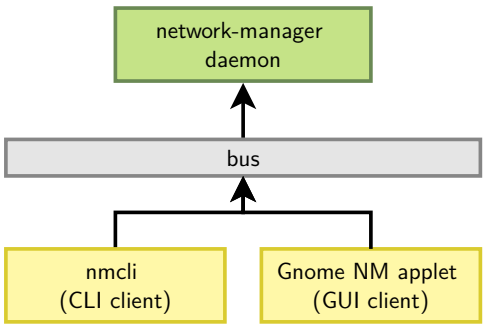


© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



D-Bus

- *Message-oriented middleware mechanism that allows communication between multiple processes running concurrently on the same machine*
- Relies on a daemon to pass messages between applications
- Mainly used by system daemons to offer services to client applications
- Example: a network configuration daemon, running as *root*, offers a D-Bus API that CLI and GUI clients can use to configure networking
- Several busses
  - One system bus, accessible by all users, for system services
  - One session bus for each user logged in
- Object model: interfaces, objects, methods, signals
- https://www.freedesktop.org/wiki/Software/dbus/





systemd (1)

- Modern *init* system used by almost all Linux desktop/server distributions
- Much more complex than *BusyBox init*, but also much more powerful
- Only supported with *glibc*, not with *uClibc* and *Musl*
- Provides features such as
  - Parallel startup of services, taking into account dependencies
  - Monitoring of services
  - On-demand startup of services, through *socket activation*
  - Resource-management of services: CPU limits, memory limits
- Configuration based on *unit files*
  - Declarative language, instead of shell scripts used in other init systems



systemd (2)

- Systemd also provides
  - *journald*, logging daemon, replacement for *syslogd*
  - *networkd*, network configuration management
  - *udevd*, hotplugging and `/dev` management
  - *logind*, login management
  - *systemctl*, tool to control/monitor systemd
  - And many, many other things
- https://systemd.io/



systemd service unit file example

/usr/lib/systemd/system/sshd.service

```
[Unit]
Description=OpenSSH server daemon
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target sshd-keygen.service
Wants=sshd-keygen.service

[Service]
EnvironmentFile=/etc/sysconfig/sshd
ExecStart=/usr/sbin/sshd -D $ OPTIONS
ExecReload=/bin/kill -HUP $ MAINPID
KillMode=process
Restart=on-failure
RestartSec=42s

[Install]
WantedBy=multi-user.target
```

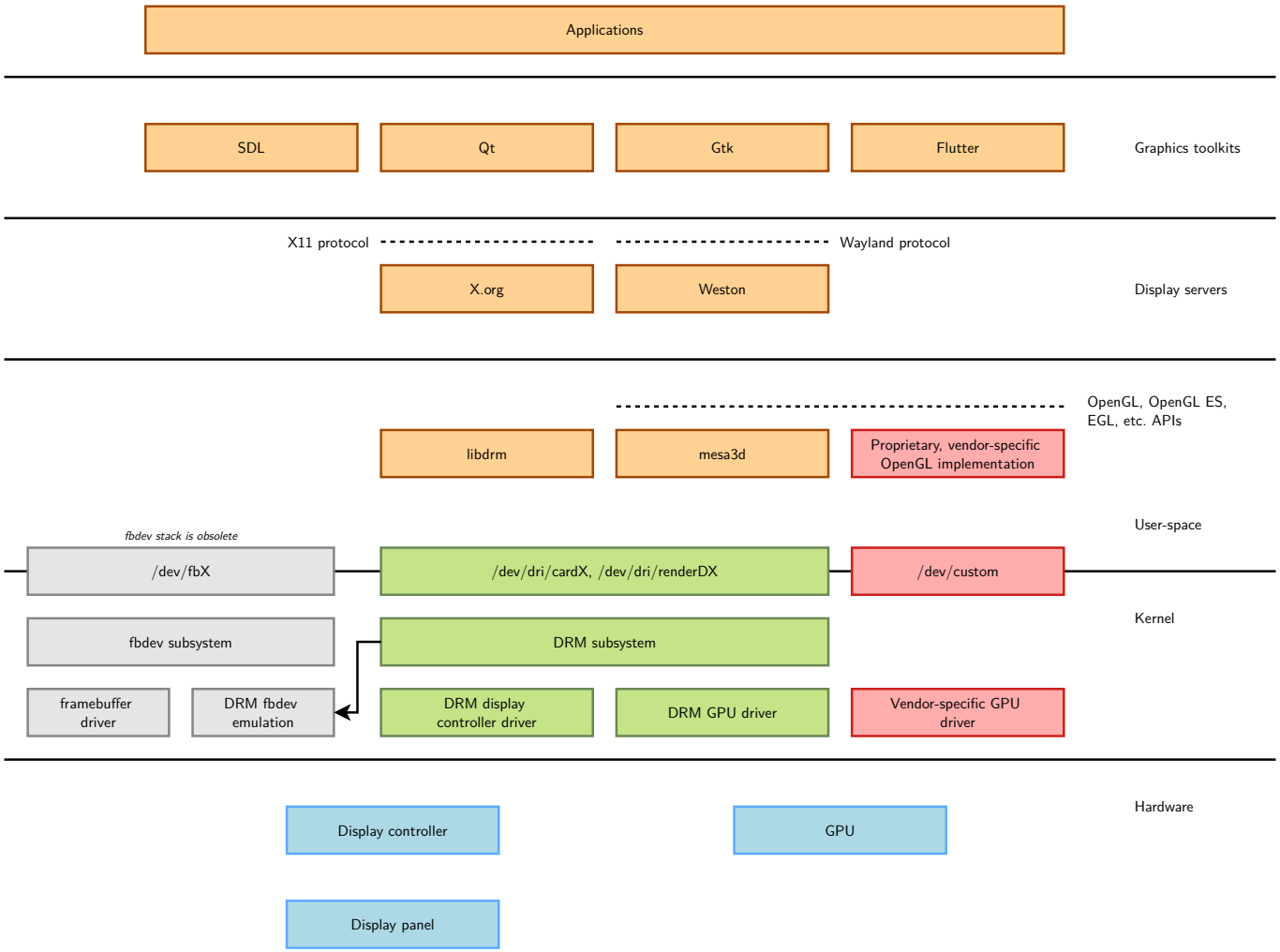


Example systemctl/journalctl commands

- `systemctl status`, status of all services
- `systemctl status <service>`, status of one service
- `systemctl [start|stop] <service>`, start or stop a service
- `systemctl [enable|disable] <service>`, enable or disable a service, i.e. whether it should start at boot time
- `systemctl list-units`, list all available units
- `journalctl -a`, all logs
- `journalctl -f`, show the last entries, and keep printing new entries as they arrive
- `journalctl -u`, logs from a particular service



Linux graphics stack overview





Display controller support

- Deprecated Linux kernel subsystem: *fbdev*
  - Still a few old graphics drivers only available in this subsystem
  - If possible, don't use!
  - https://en.wikipedia.org/wiki/Linux_framebuffer
- Modern Linux kernel subsystem: *DRM*
  - Supports display controllers of SoC or graphics cards, and all types of display panels and bridges: parallel, LVDS, DSI, HDMI, DisplayPort, etc.
  - Also supports small display panels connected over I2C or SPI
  - Devices exposed as `/dev/dri/cardX`
  - Companion user-space library: `libdrm`, includes a very handy test tool: `modetest`
  - https://en.wikipedia.org/wiki/Direct_Rendering_Manager



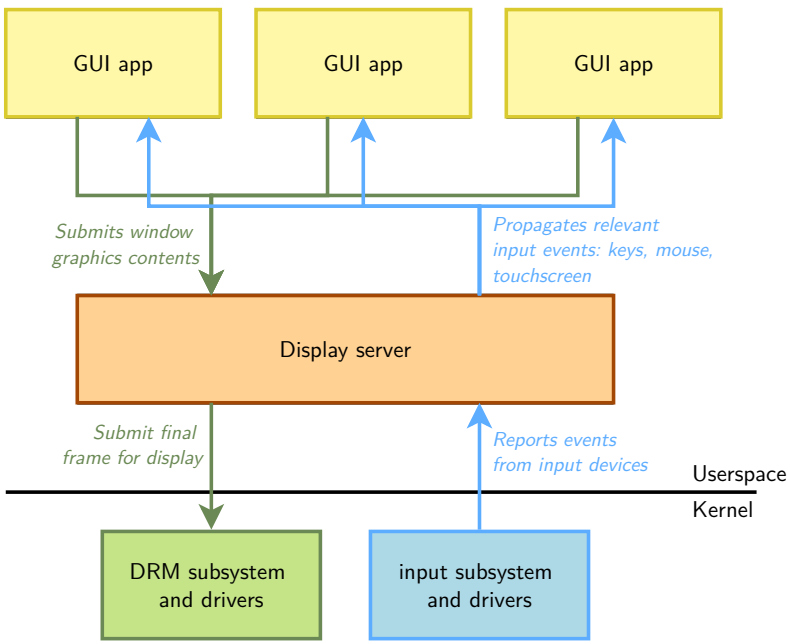
GPU support: OpenGL acceleration

- Open-source
  - A kernel driver in the DRM subsystem to send commands to the GPU and manage memory
  - `mesa3d` user-space library implementing the various OpenGL APIs, contains massive GPU-specific logic
  - More and more GPUs supported
  - https://www.mesa3d.org/
- Proprietary
  - Many embedded GPUs used to be supported only through proprietary blobs → long-term maintenance issues
  - A kernel driver provided out-of-tree by the vendor → they are not accepted upstream if the user-space is closed source
  - A (huge) closed-source user-space binary blob implementing the various OpenGL APIs



Concept of display servers

- The Linux kernel does not handle the *multiplexing* of the display and input devices between applications
  - Only one user-space application can use a display and a given set of input devices
- Display servers are special user-space applications that multiplex display/input by:
  - Allowing multiple client GUI applications to submit their window contents
  - Composing the final frame visible on the screen, based on contents submitted by applications, window visibility and layering
  - Propagating input events to the appropriate clients, based on focus





X11 and X.org

- *X.org* is the historical display server on UNIX systems, including Linux
- Implements the *X11* protocol, used between clients and the server
  - UNIX socket for local clients, TCP for remote clients
- On modern Linux, works on top of DRM or fbdev for graphics, input subsystem for input events
- Still maintained, but now legacy.
- X11 license
- https://www.x.org





Wayland

- *Communication **protocol** that specifies the communication between a display server and its clients, as well as a C library implementation of that protocol*
- A display server using the Wayland protocol is called a Wayland **compositor**
- Modern replacement for the aging X11 protocol
- More heavily based on OpenGL technologies
- https://wayland.freedesktop.org/
- https://en.wikipedia.org/wiki/Wayland_(display_server_protocol)





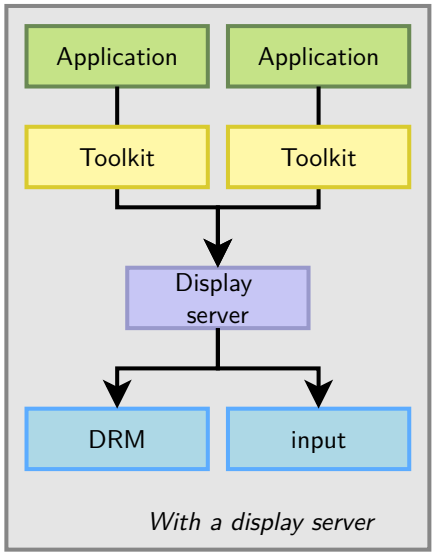
Wayland compositors

- Weston
  - The reference compositor
  - https://gitlab.freedesktop.org/wayland/weston
- Mutter, used by the GNOME desktop environment https://gitlab.gnome.org/GNOME/mutter
- wlroots, a Wayland compositor library, used by
  - Cage, a Wayland kiosk-style compositor https://github.com/Hjdskes/cage
  - swayWM, a tiling Wayland compositor https://swaywm.org/
- And many more https://wiki.archlinux.org/title/wayland#Compositors



Concept of graphics toolkits

- The X11 and Wayland protocols are very low-level protocols
- While possible, developing applications directly using those protocols or their corresponding client libraries would be painful
- Existence of *toolkits*
  - Some of them work only on top of a display server: X11 or Wayland
  - Some of them can work directly on top of DRM + input, for single full-screen applications
- Widget-oriented toolkits, with APIs to create windows, buttons, text fields, drop-down lists, etc.
- Game/multimedia-oriented toolkits, with no pre-defined widget API





Qt

- Highly popular and well-documented development framework, providing:
  - Core libraries: data structures, event handling, XML, databases, networking, etc.
  - Graphics libraries: widgets and more
- Standard API is C++, but bindings to other languages available
- Works as
  - Single application with DRM with OpenGL, or *fbdev* with no acceleration
  - Multiple applications on top of X11 or Wayland
- Multiplatform: Linux, MacOS, Windows.
- Somewhat complex licensing, with a mix of LGPLv3, GPLv2, GPLv3, and an (expensive) commercial license
- https://www.qt.io/





Gtk

- Toolkit used as the base for the GNOME desktop environment, the most popular desktop environment for Linux desktop distributions, but losing traction in embedded projects.
- Composed of *glib* (core library), *pango* (text handling), *cairo* (vector graphics), *gtk* (widget library)
- Standard API in C, but bindings exist for many languages
- Requires a display server: X11 or Wayland
- License: LGPLv2
- Version 3.x the most deployed currently, 4.x is a new major release
- Multiplatform: Linux, MacOS, Windows.
- https://www.gtk.org

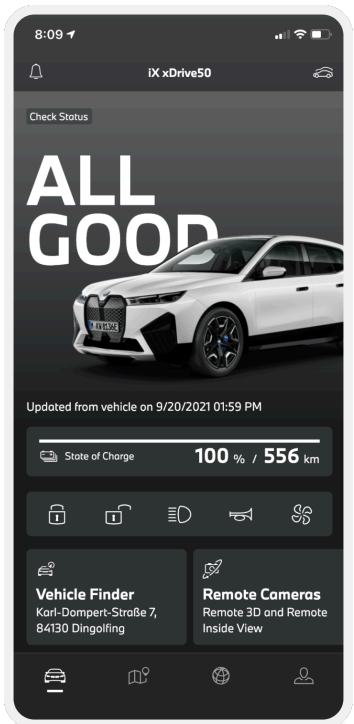


Flutter

- Cross-platform UI application development: Linux, Android, iOS, Windows, MacOS
- Developed and maintained by Google
- Applications must be developed using the *Dart* programming language
- Applications can run in the Dart virtual machine, or be natively compiled for better performance.
- License: BSD-3-Clause
- https://flutter.dev

Read our blog post: https://bootlin.com/blog/flutter-nvidia-jetson-openembedded-yocto/

Flutter





SDL

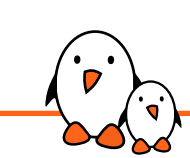
- *Cross-platform development library designed to provide low level access to audio, keyboard, mouse, joystick, and graphics hardware*
- Implemented in C, lightweight
- Does not provide a widget library
- Games, media players, custom UIs
- License: zlib license (simple permissive license)



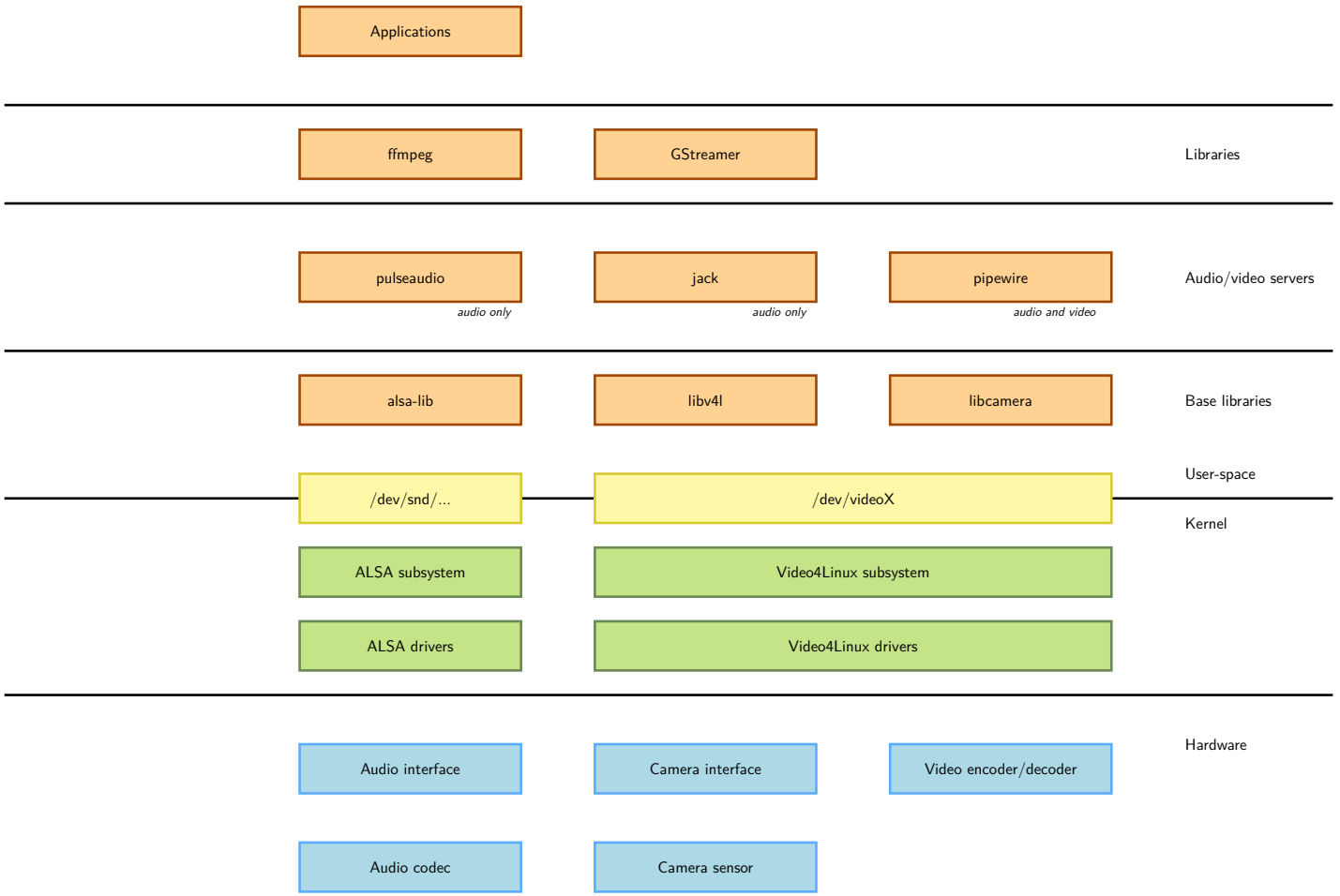


Other graphical toolkits

- Enlightenment Foundation Libraries (EFL) / Elementary
  - Lightweight and very powerful, but a lot less popular
  - Work on top of X or Wayland.
  - License: LGPLv2.1
  - https://www.enlightenment.org/about-efl.md
- LVGL
  - Very lightweight, mostly targeted at micro-controllers, but also runs on Linux
  - License: MIT
  - https://lvgl.io/
- See https://en.wikipedia.org/wiki/List_of_widget_toolkits



Linux multimedia stack overview





Audio stack

- Kernel-side: the ALSA subsystem, *Advanced Linux Sound Architecture*
  - Includes drivers for audio interfaces and audio codecs
  - Exposes audio devices in `/dev/snd/`
  - https://alsa-project.org
- Companion user-space library: *alsa-lib*
- Audio servers
  - Needed when multiple applications share audio devices: mix audio stream, route audio stream from specific applications to specific devices
  - *JACK*: mainly for professional audio
  - *pulseaudio*: mainly for regular desktop Linux audio
  - *pipewire*: modern replacement for both pulseaudio and JACK, already adopted by some Linux distributions
  - https://pipewire.org/



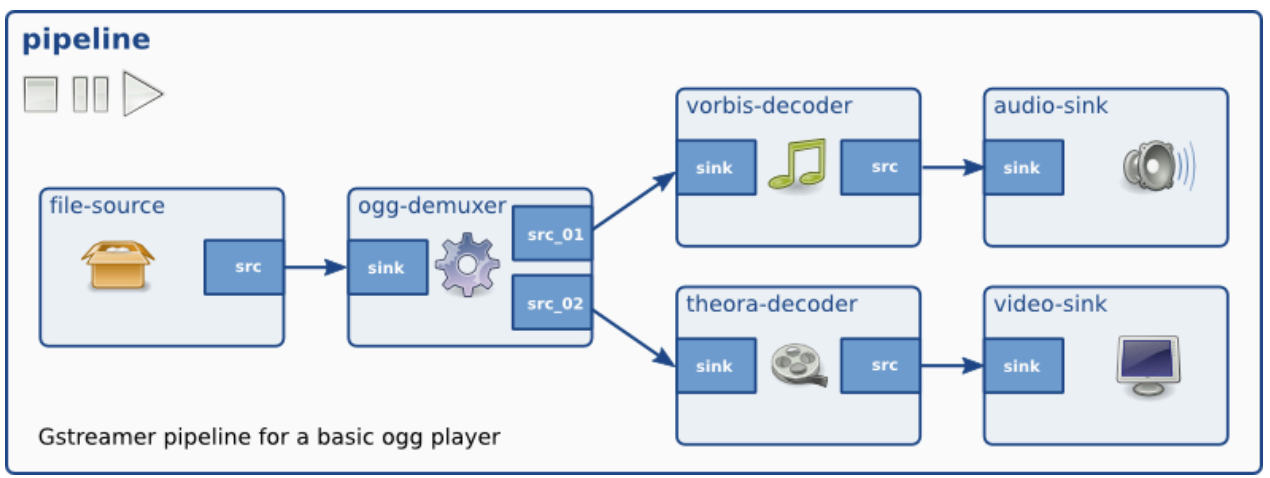
Video stack

- Kernel-side: Video4Linux subsystem, or V4L in short
  - Supports camera devices: webcams as well as camera interfaces of SoCs and camera sensors (parallel, CSI, etc.)
  - Also used to support video encoding/decoding HW accelerators: H264, H265, etc.
  - Exposes video devices as `/dev/videoX`
  - https://www.linuxtv.org/
- Traditional user-space library: *libv4l*
- New user-space library, more modern, with many more features, under adoption: *libcamera*
- Supported in lots of multimedia stacks/software: GStreamer, ffmpeg, VLC, etc.



GStreamer

- *Library for constructing graphs of media-handling components*
- Allows to create *pipelines* to transform, convert, stream, display, capture multimedia streams, both audio and video
- Composed of a vast amounts of plugins: video capture/display, audio capture/playback, encoding/decoding, scaling, filtering, and more.
- https://gstreamer.freedesktop.org/
- An interesting alternative is *ffmpeg*

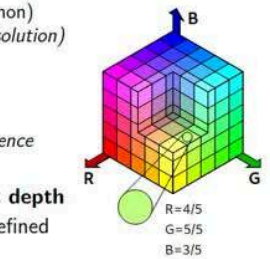


Further details on Linux graphics and multimedia stacks

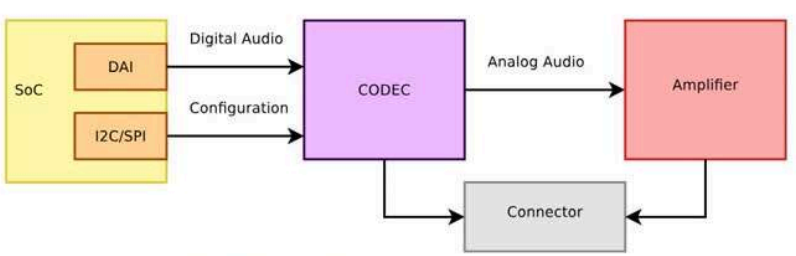
- Bootlin's *Understanding the Linux graphics stack* training
- Bootlin's *Embedded Linux Audio* training
- Complete courses focused exclusively on those topics
- Freely available training materials

Color quantization approaches

- Different approaches exist for **color quantization**:
  - **Uniform** quantization in the color range (most common)
    values are attributed to colors with a regular step (resolution)
  - **Irregular** quantization with indexed colors (palettes)
    values are attributed to colors as needed
- Uniform color coordinates are quantized with:
  - A given **resolution**: *the smallest possible color difference*
  - A given **range**: *the span of representable colors*
- A given number of bits are used for quantization: **bit depth**
- A **trade-off** between range and resolution must be defined
  - Increasing the resolution reduces the range
  - Increasing the range reduces the resolution

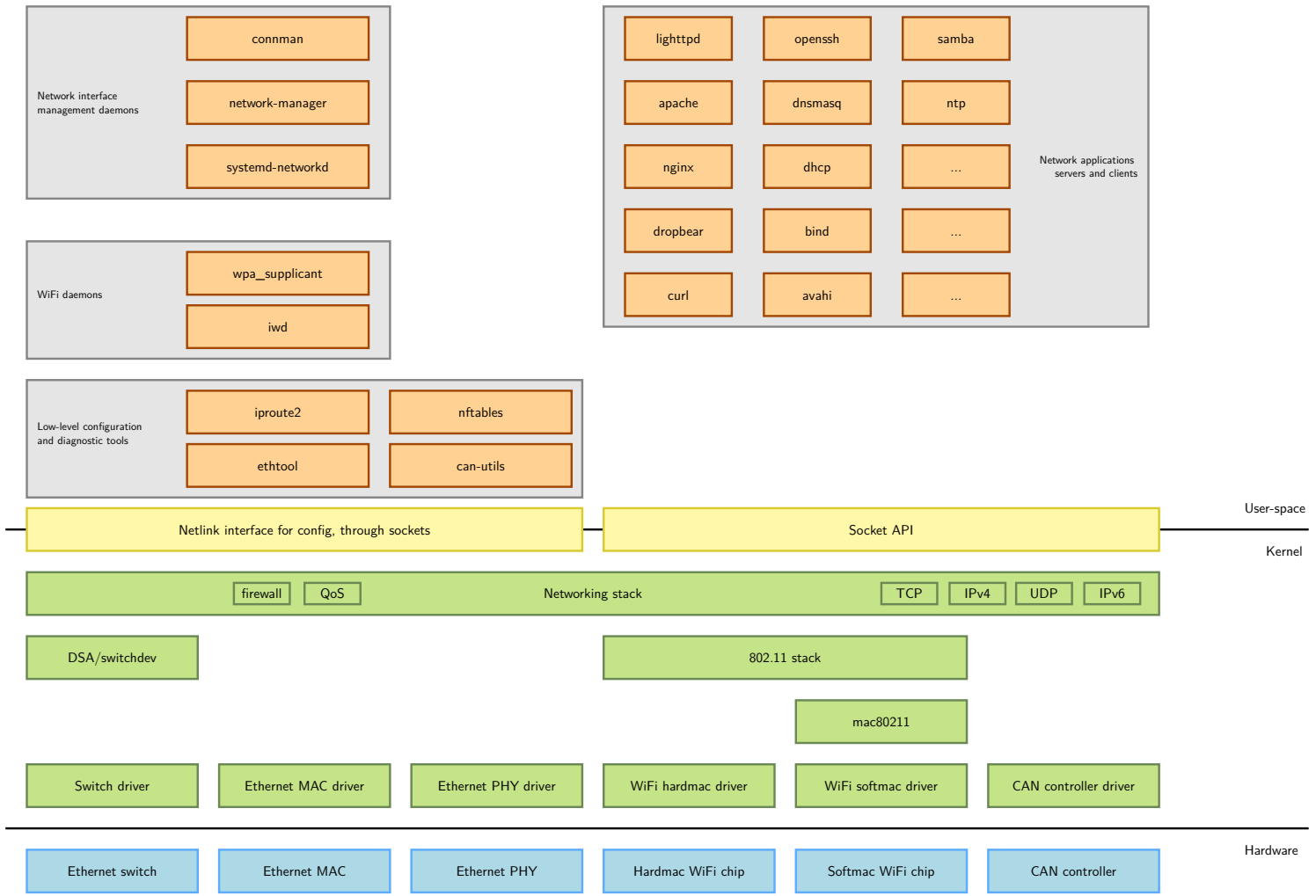


Anatomy



Example of an embedded system sound card

Linux networking stack





Web accessible UI

- Very common in embedded systems to use a Web interface for device configuration/ monitoring
- Needs a web server: *BusyBox httpd* for very simple needs, *lighttpd*, *nginx*, *apache* for more complex needs
- Can use PHP, NodeJS or other interpreted languages, or simple CGI shell scripts



Web browsers: rendering engines

To add HTML rendering capability to your device

- WebKit
  - Started by Apple, used in iOS, Safari
  - Open source project: LGPLv2.1 and BSD-2-Clause
  - https://webkit.org/
  - Integrated with Gtk: WebKitGTK
  - Integrated with Qt: QtWebKit
  - Port optimized for embedded devices: WPE WebKit
- Blink
  - Forked from WebKit
  - Developed by Google, used in Chrome
  - https://en.wikipedia.org/wiki/Blink_(browser_engine)
  - Integrated with Qt: QtWebEngine
  - Used by Electron



Web-based UIs

- An alternative to native GUI applications is to create a GUI based on Web technologies
- Run a Web browser full-screen, and use popular Web technologies to develop the application
- Some possible options
  - *Cog*, a simple launcher for the WPE Webkit port
  - *Electron*, a way to package a NodeJS application with a web rendering engine, into a self-contained application
- Beware of the footprint and performance impact: a web rendering engine is a massive and resource-consuming piece of software



Programming languages

- Wide range of languages and frameworks available, not just C/C++
- Beware of footprint and performance implications
- Natively compiled languages
  - Rust
  - Go
  - Ada
  - Fortran
- Interpreted languages
  - Python
  - Javascript, NodeJS
  - Lua
  - Shell scripts
  - Perl, Ruby, PHP

Practical lab - Integration of additional software stacks



- Integration of *systemd* as an init system
- Use *udev* built in *systemd* for automatic module loading